TanStack Query + Zustand：从零到一的 React 状态管理教程

目标读者： 会写基本 React 组件、知道 `useState`、能读懂 TypeScript 基础语法，但还没有系统掌握服务端状态与客户端状态的开发者。

文档版本： 按当前官方 TanStack Query React v5 文档与 Zustand 官方文档编写。本文以 React + TypeScript 为例；部分安装命令和代码可按项目使用的包管理器调整。

学完以后，你应该真正会什么

你将从一个具体的任务管理页面出发，先把服务端数据读出来，再学习缓存与重新验证，随后完成创建、修改、乐观更新、分页和预取，最后用 Zustand 管理筛选条件、选中项、侧栏状态和持久化偏好。最终，你能回答一个关键问题：**为什么任务数据不能再复制一份到 Zustand?**

学习时请跟着每章的代码理解“数据从哪里来、谁拥有它、什么时候会改变、改变后谁负责让其他组件知道”。这比背 API 名称更重要。

章节导航

1. 先认识两种完全不同的状态
2. 创建项目，并让 Query 正式工作
3. Query key、queryFn 与 useQuery 的完整工作方式
4. 缓存的时间观念：staleTime、gcTime 与刷新
5. useMutation：把写操作接入缓存生命周期
6. 取消请求，以及为什么这对乐观更新很重要
7. 乐观更新：先改缓存，失败再回滚
8. 分页、无限滚动、依赖查询和预取
9. 结构共享、引用稳定性与 Devtools
10. Zustand 入门：store、action 与 selector
11. 把 Zustand 状态和 Query 连接起来，但不互相复制
12. Zustand persist：哪些状态值得留下来?
13. 测试与调试：让每个用例都独立
14. 用一张规则表收束整套心智模型

第 1 章：先认识两种完全不同的状态

假设我们在写一个任务管理页面。页面需要展示服务器上的任务，还需要允许用户打开侧栏、选中一条任务、切换筛选条件。

看起来这些东西都是“状态”，但它们的生命周期不同。

服务端状态（server state） 是服务器持有、客户端只是读取或修改副本的数据。它可能被其他用户、其他浏览器标签页、定时任务或后台服务改动。客户端需要处理请求、缓存、过期、重新获取、重试、写入失败以及与服务器对账。例如任务列表、任务详情、商品库存、用户资料。

客户端 UI 状态（client/UI state） 是当前应用交互本身的状态。它通常由用户操作驱动，服务器不一定知道它。例如侧栏展开与否、临时弹窗、正在编辑的任务 ID、本地排序选择、未提交的筛选器草稿。

先做一个划分：

页面需求	推荐归属	说明
服务器上的任务列表和任务详情	TanStack Query	有缓存、后台刷新、错误和请求生命周期
“所有 / 未完成 / 已完成”的筛选条件	组件状态、URL 或 Zustand	它是查询条件，不是任务数据
侧栏展开状态	Zustand 或组件状态	纯 UI 状态，按是否需要跨组件共享来选择
当前选中的任务 ID	Zustand 或组件状态	只保存 ID；详情仍通过 Query 获取
创建任务时的请求状态	`useMutation`	提交过程、成功、错误、重试规则都由 mutation 管理
任务列表的服务器返回结果	Query 缓存	不要再完整复制到 Zustand

为什么不该把它们混成同一种状态？

如果把任务列表请求后写入 Zustand，常见代码会变成这样：先发请求，再把结果存到 Zustand；过一会儿又要考虑缓存多久、组件重新挂载是否发请求、窗口重新聚焦是否刷新、用户修改任务后如何更新每一个副本、失败怎样回滚、不同筛选条件怎么区分缓存。你会逐步在 Zustand 中手工重造一套不完整的服务端缓存系统。

如果反过来，把侧栏开关、临时弹窗、当前鼠标拖拽目标等 UI 状态塞进 Query 缓存，则会把本地交互状态与异步请求、缓存新鲜度和服务器数据同步绑在一起，徒增复杂度。

请记住以下规则：

1. **服务器数据的权威来源是服务器；Query 缓存是它在客户端的可复用副本。**
2. **Zustand 保存客户端自己的状态，不负责复制 Query 数据。**
3. 修改服务器数据时，通过 mutation 处理写入，再更新或失效 Query 缓存。
4. 如果 UI 状态决定了服务器查询结果，例如 `filter`、`page`、`search`、`tenantId`，这些变量必须反映在 query key 中。

本教程的学习项目

后面的例子围绕任务管理应用。假设后端提供以下接口：

- `GET /api/tasks?status=all&page=1&q=design`：读取任务分页列表。
- `GET /api/tasks/:id`：读取某个任务详情。
- `POST /api/tasks`：创建任务，返回创建好的任务对象。
- `PATCH /api/tasks/:id`：修改任务，返回修改后的任务对象。
- `DELETE /api/tasks/:id`：删除任务。

这是教程中的后端契约，不是指某个公开演示 API 一定存在这些接口。跟着学习时可以把 URL 换成你自己的后端。

第 2 章：创建项目，并让 Query 正式工作

如果你已经有 React + TypeScript 项目，可以直接跳到配置部分。新项目可以用 Vite：

```
npm create vite@latest state-lab -- --template react-ts
cd state-lab
npm install
npm install @tanstack/react-query zustand
npm install -D @tanstack/react-query-devtools
npm run dev
```

2.1 创建 QueryClient

TanStack Query 的核心对象是 `QueryClient`。可以把它理解为应用内的“服务端数据缓存协调中心”：它管理 query 缓存、失效、预取、取消、全局默认选项等。React 组件通过 `QueryClientProvider` 获得它。

在 `src/AppProviders.tsx` 中写：

```
import { useState, type PropsWithChildren } from 'react'
import {
  QueryClient,
  QueryClientProvider,
} from '@tanstack/react-query'
import { ReactQueryDevtools } from '@tanstack/react-query-devtools'

export function AppProviders({ children }: PropsWithChildren) {
  // 让同一个应用实例复用同一个 QueryClient，而不是每次渲染都重建。
  const [queryClient] = useState(
    () =>
      new QueryClient({
        defaultOptions: {
          queries: {
            staleTime: 30_000,
            // 这里设成 30 秒便于教程演示；不设置时默认值是 5 分钟。
            gcTime: 5 * 60_000,
          },
        },
      }),
  )

  return (
    <QueryClientProvider client={queryClient}>
      {children}
      {import.meta.env.DEV && <ReactQueryDevtools initialIsOpen={false} />}
    </QueryClientProvider>
  )
}
```

然后在 src/main.tsx 中包裹应用：

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App'
import { AppProviders } from './AppProviders'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <AppProviders>
      <App />
    </AppProviders>
  </StrictMode>,
)
```

这里刻意自定义了 staleTime，是为了让初学者更容易观察缓存新鲜度。请勿误以为这就是库的默认设置：**TanStack Query 的默认 staleTime 是 0**。如果使用 SSR，每个请求也要按照框架的 SSR 指引管理 QueryClient，不能把某个用户的服务端缓存跨请求共享给另一个用户。

2.2 一个最小的请求函数

浏览器的 fetch 在网络失败时通常会 reject，但遇到 HTTP 404 或 500 时不会自动抛错。因此 Query 的请求函数应该检查 response.ok，并在失败时抛出错误。

在 src/lib/http.ts 中写：

```
export async function fetchJson<T>(
  url: string,
  init?: RequestInit,
): Promise<T> {
  const response = await fetch(url, init)

  if (!response.ok) {
    const detail = await response.text()
    throw new Error(detail || `HTTP ${response.status}`)
  }

  return (await response.json()) as T
}
```

该示例假设这些接口成功时都会返回 JSON。如果某个删除接口返回 204 No Content，就不要直接用这个函数解析它；应单独写一个不调用 response.json() 的请求函数。

现在我们具备两个基础：一个所有组件共享的 QueryClient，以及一个出错时会正确 reject 的请求函数。下一章才开始真正缓存数据。

第 3 章：Query key、queryFn 与 useQuery 的完整工作方式

3.1 先定义数据形状

在 src/features/tasks/taskTypes.ts 中：

```ts
export type TaskStatus = 'all' | 'open' | 'done'

export interface Task {
  id: string
  title: string
  completed: boolean
  updatedAt: string
}

export interface TaskListParams {
  status: TaskStatus
  page: number
  q: string
}

export interface TaskPage {
  items: Task[]
  page: number
  totalPages: number
  hasMore: boolean
}

export interface CreateTaskInput {
  title: string
}

export interface UpdateTaskInput {
  id: string
  patch: Partial<Pick<Task, 'title' | 'completed'>>
}
```

3.2 Query key 就是缓存身份

Query key（查询键）不是随便填的标签。它决定哪一个请求结果存在哪个缓存条目里；相同 key 的观察者可以共享缓存，key 改变时 Query 会观察新的缓存条目并按需要获取数据。

我们要支持“任务列表”和“单条任务详情”，还要支持不同的筛选、页码和搜索词。可以用分层 key 工厂：

```ts
// src/features/tasks/taskKeys.ts
import type { TaskListParams } from './taskTypes'

export const taskKeys = {
  all: ['tasks'] as const,
  lists: () => [...taskKeys.all, 'list'] as const,
  list: (params: TaskListParams) =>
    [...taskKeys.lists(), params] as const,
  details: () => [...taskKeys.all, 'detail'] as const,
```

```
  detail: (id: string) => [...taskKeys.details(), id] as const,
}
```

这个层级产生了很实用的匹配能力：

- `['tasks']` 可以匹配所有任务相关查询。
- `['tasks', 'list']` 可以匹配所有任务列表缓存，包括不同页码和筛选条件。
- `['tasks', 'list', { status: 'open', page: 1, q: '' }]` 只代表一个具体查询条件对应的列表缓存。
- `['tasks', 'detail', 'task-123']` 代表 ID 为 task-123 的详情缓存。

Query key 应由可序列化的数据组成。对象属性顺序会进行稳定哈希处理，但数组元素的顺序有意义。更重要的是，key 中必须包含所有会影响请求结果的变量。

例如，若 queryFn 根据 status、page、q 读取不同列表，缓存 key 也必须包含这三个参数。漏掉某个参数会让不同请求错误地共用同一缓存，出现“明明切了筛选条件却展示旧结果”的问题。对于多租户或用户权限相关数据，租户 / 用户上下文也必须能区分相应缓存；不要把密码或访问令牌本身放入 key。

3.3 编写 API 函数

```
// src/features/tasks/taskApi.ts
import { fetchJson } from '../../lib/http'
import type {
  CreateTaskInput,
  Task,
  TaskListParams,
  TaskPage,
  UpdateTaskInput,
} from './taskTypes'

export function getTasks(
  params: TaskListParams,
  signal?: AbortSignal,
): Promise<TaskPage> {
  const search = new URLSearchParams({
    status: params.status,
    page: String(params.page),
    q: params.q,
  })

  return fetchJson<TaskPage>(`/api/tasks?${search}`, { signal })
}

export function getTask(
  id: string,
  signal?: AbortSignal,
): Promise<Task> {
  return fetchJson<Task>(`/api/tasks/${encodeURIComponent(id)}`, { signal })
}

export function createTask(input: CreateTaskInput): Promise<Task> {
  return fetchJson<Task>('/api/tasks', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(input),
```

```
  })
}

export function updateTask(input: UpdateTaskInput): Promise<Task> {
  return fetchJson<Task>(`/api/tasks/${encodeURIComponent(input.id)}`, {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(input.patch),
  })
}

export async function deleteTask(id: string): Promise<void> {
  const response = await fetch(`/api/tasks/${encodeURIComponent(id)}`, {
    method: 'DELETE',
  })
  if (!response.ok) throw new Error(`删除失败：HTTP ${response.status}`)
}
```

将 API 函数放在组件外有几个好处：组件更容易阅读，业务请求可复用，也更容易单独测试。`signal` 参数则为下一阶段的请求取消留好了接口。

3.4 第一个 useQuery

```
// src/features/tasks/useTasks.ts
import { useQuery } from '@tanstack/react-query'
import { getTasks } from './taskApi'
import { taskKeys } from './taskKeys'
import type { TaskListParams } from './taskTypes'

export function useTasks(params: TaskListParams) {
  return useQuery({
    queryKey: taskKeys.list(params),
    queryFn: ({ signal }) => getTasks(params, signal),
  })
}
```

`useQuery` 的核心参数就两个：

- `queryKey`：当前数据的身份与缓存地址。
- `queryFn`：返回 Promise 的异步函数，成功时返回数据，失败时抛出错误。

这两个参数共同描述“组件需要哪一份异步数据，以及如何取得它”。不要在 `queryFn` 中吞掉错误后返回 `undefined`；Query 需要通过成功或失败的 Promise 来驱动状态。

3.5 loading、error、success 并不是只有一个布尔值

```
function TaskList({ params }: { params: TaskListParams }) {
  const query = useTasks(params)

  if (query.isPending) {
    return <p>正在准备任务数据……</p>
  }

  if (query.isError && !query.data) {
```

```
    return <p role="alert">加载失败：{query.error.message}</p>
  }

  // 到这里通常已有可展示的数据。
  return (
    <section>
      {query.isFetching && <small>正在后台同步……</small>}
      {query.isRefetchError && (
        <p role="alert">刷新失败，当前仍显示上一次成功取得的数据。</p>
      )}
      <ul>
        {query.data?.items.map((task) => (
          <li key={task.id}>{task.title}</li>
        ))}
      </ul>
    </section>
  )
}
```

在 v5 中，先理解三个主要状态：

- `isPending` / `status === 'pending'`：当前 query 还没有成功数据。依赖查询在 `enabled: false` 时也可能处于 pending，因此它不完全等同于“网络正在加载”。
- `isError` / `status === 'error'`：最近一次尝试处于错误状态；可以通过 `error` 读取错误对象。
- `isSuccess` / `status === 'success'`：查询成功，`data` 可用。

另有一条独立的请求状态轴：`fetchStatus` / `isFetching`。`isFetching` 表示现在有请求正在执行，包括首次请求和后台重新获取。页面已有数据时，后台请求也可能进行，因此不要一看到 `isFetching` 就把整个页面换成全屏 loading。

`isLoading` 在 v5 中表示 `isPending && isFetching`，更接近“首次正在加载”。常见做法是：首次没有数据时展示 loading；没有可用数据且失败时展示错误页；已有数据时继续展示内容，并用小型提示表现后台刷新。

本章动手练习

1. 故意把列表接口改成 `/api/does-not-exist`，观察 `isError` 和 `error.message`。
2. 在两个组件中都调用 `useTasks({ status: 'all', page: 1, q: '' })`，观察同一 key 的缓存如何共享。
3. 先将 `page` 从 key 中删掉，再切换页码，思考为什么缓存身份就错了；之后务必恢复。

第 4 章：缓存的时间观念——staleTime、gcTime 与刷新

这一章是理解 TanStack Query 的关键。新手最容易把“缓存了多久”和“数据是否新鲜”误认为同一个概念。

4.1 staleTime：多长时间内认为数据新鲜

`staleTime` 控制数据在多长时间内被视为 fresh（新鲜）。例如 `staleTime: 30_000` 表示成功获取数据后 30 秒内，Query 将它视为新鲜数据。时间过去之后它变成 stale（过期 / 陈旧），但**变 stale 并不等于马上自动发送请求**。

stale 的意思是：下次遇到自动重新验证的触发点时，它可以被重新获取。常见触发点是新观察者挂载、窗口重新聚焦、网络重新连接，或者显式失效；此外，`refetchInterval` 可以独立设置轮询。

```
useQuery({
  queryKey: taskKeys.list(params),
  queryFn: ({ signal }) => getTasks(params, signal),
  staleTime: 60_000, // 1 分钟内视为新鲜
})
```

适合的值取决于业务：变化频繁的实时状态可以设短一些；更新不频繁的分类或配置可以更长。不要为了“少发请求”就一律设置成很长，也不要因为怕旧数据而把它永远设成 0。应根据“数据变化的可能性”和“用户能接受多旧的数据”决定。

4.2 gcTime：没有组件使用时，缓存还保留多久

gcTime 控制的是 inactive（非活跃、没有观察者）的缓存可以在内存中保留多久。默认通常是 5 分钟。超过这个时间，该缓存才会被垃圾回收。

```
useQuery({
  queryKey: taskKeys.detail(taskId),
  queryFn: ({ signal }) => getTask(taskId, signal),
  staleTime: 30_000,
  gcTime: 10 * 60_000,
})
```

设想你打开任务 A 的详情，再离开页面：

1. 在缓存仍新鲜的期间返回，通常可直接使用缓存，不需要重新请求。
2. 数据变 stale 后返回，可能先展示已有数据，然后后台重新请求。
3. 如果页面一直没人使用该 query，过了 gcTime，缓存会被清理；之后再回来就需要重新获取数据。

因此，staleTime 是新鲜度策略，gcTime 是非活跃缓存的保留策略。它们彼此独立。gcTime 不是“多少时间后自动刷新”。

4.3 invalidateQueries：写操作之后让缓存失效

当服务器上的任务被创建或修改时，客户端可能不知道哪些列表受到影响。invalidateQueries 是常用的同步办法：它把匹配的 query 标记为 stale，并默认对匹配的活跃 query 触发后台重新获取。

```
await queryClient.invalidateQueries({
  queryKey: taskKeys.lists(),
})
```

利用分层 key，可以只失效所有列表，不影响详情：

```
// 所有任务列表（任意页码、筛选条件）
queryClient.invalidateQueries({ queryKey: taskKeys.lists() })

// 所有任务相关缓存（列表与详情）
queryClient.invalidateQueries({ queryKey: taskKeys.all })

// 只匹配某个具体 key
queryClient.invalidateQueries({
  queryKey: taskKeys.detail(taskId),
  exact: true,
})
```

默认会重新获取匹配的活跃 query；不活跃 query 通常只是变 stale，等再次被使用时再获取。可以通过 `refetchType` 调整行为。失效不是删除缓存；它表达的是“已有数据可能过期，需要重新验证”。

4.4 主动 refetch 与自动刷新有什么区别

`useQuery` 返回的 `refetch()` 是对当前查询主动请求一次：

```
const query = useTasks(params)

return (
  <button onClick={() => query.refetch()} disabled={query.isFetching}>
    重新加载
  </button>
)
```

它与 `invalidateQueries` 语义不同：

- `refetch()`：我现在就想重新请求当前这个查询。
- `invalidateQueries()`：缓存内容可能过时；让匹配数据失效，并按默认规则重新验证活跃 query。

Query 的常见默认行为（客户端）需要熟悉：

- `staleTime` 默认是 0，拿到数据后立即被视为 stale。
- stale query 在新观察者挂载、窗口重新聚焦、网络重新连接等事件时会自动后台重新获取（具体可用 `refetchOnMount`、`refetchOnWindowFocus`、`refetchOnReconnect` 配置）。
- `refetchInterval` 可配置定时轮询，并独立于 `staleTime`。
- 非活跃缓存默认约 5 分钟后 GC。
- query 默认会在客户端失败后重试 3 次，采用指数退避延迟。
- JSON 兼容的数据默认启用结构共享，以尽量复用没有变化部分的引用。

这也是为什么开发时切到浏览器 DevTools 再切回来，可能会看到请求再次发生：窗口聚焦是默认的自动重新验证触发点之一。多数情况下不用禁用它；先确认这是预期行为，再为特定 query 调整配置。

本章动手练习

1. 在一个 query 上设置 `staleTime: 60_000`，重复进入页面并在 1 分钟内切换浏览器标签，观察行为。
2. 将 `gcTime` 调短，离开页面等待后再回来，观察缓存被清理的结果。
3. 创建任务后只失效 `taskKeys.lists()`，解释为什么比清空整个 QueryClient 更合适。

第 5 章：useMutation——把“操作”接入存生命周期

`useQuery` 通常用于读取 / 获取数据；`useMutation` 用于创建、更新、删除或执行服务器副作用。Mutation 不是“一个自动缓存的数据请求”，它更像一次明确的写操作及其生命周期。

5.1 创建任务并失效列表缓存

```
// src/features/tasks/useCreateTask.ts
import { useMutation, useQueryClient } from '@tanstack/react-query'
import { createTask } from './taskApi'
import { taskKeys } from './taskKeys'
```

```ts
export function useCreateTask() {
  const queryClient = useQueryClient()

  return useMutation({
    mutationKey: ['tasks', 'create'],
    mutationFn: createTask,
    onSuccess: async (createdTask) => {
      // 返回值是服务端确认后的任务对象，可以直接更新详情缓存。
      queryClient.setQueryData(
        taskKeys.detail(createdTask.id),
        createdTask,
      )

      // 创建可能改变各个筛选列表和分页总数，因此刷新相关列表。
      await queryClient.invalidateQueries({
        queryKey: taskKeys.lists(),
      })
    },
  })
}
```

组件使用：

```tsx
import * as React from 'react'

function CreateTaskForm() {
  const mutation = useCreateTask()

  async function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
    event.preventDefault()
    const formData = new FormData(event.currentTarget)
    const title = String(formData.get('title') ?? '').trim()
    if (!title) return

    mutation.mutate({ title })
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="title" aria-label="任务标题" />
      <button disabled={mutation.isPending}>
        {mutation.isPending ? '正在创建……' : '创建任务'}
      </button>
      {mutation.isError && <p role="alert">创建失败：{mutation.error.message}</p>}
      {mutation.isSuccess && <p>创建请求已成功。</p>}
    </form>
  )
}
```

真实产品里，成功提示往往应该在一段时间后消失，或由通知系统处理；示例重点是 mutation 的请求状态。

5.2 Mutation 生命周期

一个 mutation 的常见状态是：`idle`、`pending`、`success`、`error`。最常用的回调时序是：

1. onMutate：写请求发出前调用，适合做乐观更新或保存回滚快照。
2. mutationFn：真正调用服务器。
3. 如果成功，调用 onSuccess；如果失败，调用 onError。
4. onSettled：成功或失败都会调用，常用于收尾、失效缓存或释放资源。

这些回调若返回 Promise，Query 会等待该 Promise 完成再继续相应的生命周期。这意味着在 onSuccess 里 `await invalidateQueries(...)`，mutation 的 pending 状态可以覆盖到刷新操作完成为止；有时能让 UI 状态更自然，但也可能让按钮多保持一会儿禁用状态。

```
const mutation = useMutation({
  mutationFn: updateTask,
  onMutate: async (variables) => {
    // 在真正写服务器前运行
    return { taskId: variables.id }
  },
  onError: (error, variables, onMutateResult) => {
    // 失败时处理错误；onMutate 的返回值会传进来
    console.error('修改失败', onMutateResult?.taskId, error)
  },
  onSuccess: (serverTask) => {
    // 使用服务器确认后的结果
    queryClient.setQueryData(taskKeys.detail(serverTask.id), serverTask)
  },
  onSettled: async () => {
    // 无论成功还是失败都执行的收尾工作
    await queryClient.invalidateQueries({ queryKey: taskKeys.lists() })
  },
})
```

以上为 TanStack Query v5 的回调形式。onMutate 的返回值在后续回调中作为 onMutateResult 提供；回调参数中还有 mutation function context。注意不要把这几个参数位置混淆。

5.3 成功后应该失效哪些缓存？

问自己：这次写操作可能改变哪些读取结果？

- 创建任务：列表、总数、分页信息可能改变。
- 改标题：任务详情和包含此任务的列表可能改变。
- 标记完成：可能改变“全部”“未完成”“已完成”多个列表。
- 删除任务：列表、分页总数和详情缓存可能改变。

简单可靠的入门策略是：直接把服务器成功响应写进精确详情缓存，再失效受影响的列表。以后确实需要进一步优化时，再选择性地把服务器响应更新到对应列表缓存。相比手工猜测并修改每一个列表，失效后重新获取通常更简单、更安全。

5.4 错误处理与重试

应同时考虑两种错误处理：

- 数据层：让请求函数抛出错误，mutation 回调按需要回滚、记录日志或失效缓存。
- UI 层：用 isError 和 error 给用户反馈，并为可恢复错误提供重试按钮或再次提交的路径。

Query 默认客户端重试 3 次；Mutation 默认不自动重试。写操作通常不应随意重试，因为服务器可能已经成功执行，只是响应丢失。如果接口具备幂等键、版本控制或明确的幂等语义，再谨慎评估自动重试。对于 query，可自定义重试策略，例如对一般 4xx 不重试，对网络错误或部分 5xx 最多重试 2 次：

```
useQuery({
  queryKey: taskKeys.list(params),
  queryFn: ({ signal }) => getTasks(params, signal),
  retry: (failureCount, error) => {
    if (error instanceof Error && error.message.includes('HTTP 4')) {
      return false
    }
    return failureCount < 2
  },
})
```

真实应用最好定义带 `status` 的 `HttpError` 类型，而不是用字符串判断 HTTP 状态。此片段仅演示“按错误类型决定重试”的思路。

第 6 章：取消请求，以及为什么这对乐观更新很重要

用户很快切换筛选条件、离开详情页，或者正在进行的请求已经失去价值时，取消请求可以节省资源并减少过期结果覆盖新数据的风险。

TanStack Query 会把 `AbortSignal` 传给 `queryFn`。只要将它传给 `fetch`，取消信号就能传递到网络请求：

```
useQuery({
  queryKey: taskKeys.list(params),
  queryFn: ({ signal }) => getTasks(params, signal),
})
```

`getTasks` 已将 `signal` 继续传入 `fetchJson`，而 `fetchJson` 再把 `RequestInit` 传给 `fetch`，所以取消链条完整。

还可以手工取消一组查询：

```
await queryClient.cancelQueries({
  queryKey: taskKeys.lists(),
})
```

要知道一个细节：如果 `queryFn` 没有消费 `signal`，query 在组件卸载或不再使用时，底层 Promise 默认不一定会真的被取消；请求完成后的结果仍可能进入缓存。如果消费了 signal，取消会进一步中止网络请求，并使 Query 状态回到请求前的状态。使用 Suspense 专用 query hooks 时，当前取消支持有额外限制，应查阅对应文档。

为什么乐观更新前要取消正在运行的查询？想象列表后台刷新已经开始；你先在缓存中把任务标成完成；随后之前那次查询返回的旧列表又覆盖了你刚做的乐观修改。先取消相关的 in-flight query，可以减少这种覆盖窗口。

第 7 章：乐观更新——先改缓存，失败再回滚

乐观更新（optimistic update）指的是：服务器还没确认成功，界面先展示“预计会成功”的结果。如果用户点击“完成”，任务可以立即变成完成状态，同时网络请求在后台进行。失败时再恢复旧值，并给出错误反馈。

先强调：乐观更新不是每个按钮都必须用的优化。对于简单表单，等待服务器成功后再更新往往更易维护；当交互对即时反馈要求高、更新逻辑明确时，乐观更新才值得使用。

7.1 标记任务完成的基础实现

我们对所有任务列表的缓存做最小修改。由于多个不同筛选的列表都可能含有该任务，示例用 `setQueriesData` 一次更新匹配 key。

```ts
// src/features/tasks/useUpdateTask.ts
import { useMutation, useQueryClient } from '@tanstack/react-query'
import { updateTask } from './taskApi'
import { taskKeys } from './taskKeys'
import type { TaskPage, UpdateTaskInput } from './taskTypes'

export function useUpdateTask() {
  const queryClient = useQueryClient()

  return useMutation({
    mutationKey: ['tasks', 'update'],
    mutationFn: updateTask,

    onMutate: async (variables) => {
      // 1. 避免正在飞行的旧查询覆盖乐观结果
      await queryClient.cancelQueries({ queryKey: taskKeys.lists() })

      // 2. 保存所有匹配列表的快照，用于失败回滚
      const previousLists = queryClient.getQueriesData<TaskPage>({
        queryKey: taskKeys.lists(),
      })

      // 3. 只更新缓存，不直接修改旧对象
      queryClient.setQueriesData<TaskPage>(
        { queryKey: taskKeys.lists() },
        (old) => {
          if (!old) return old
          return {
            ...old,
            items: old.items.map((task) =>
              task.id === variables.id
                ? { ...task, ...variables.patch }
                : task,
            ),
          }
        },
      )

      return { previousLists }
```

```
    },

    onError: (_error, _variables, onMutateResult) => {
      // 4. 失败时恢复对应缓存的快照
      onMutateResult?.previousLists.forEach(([queryKey, data]) => {
        queryClient.setQueryData(queryKey, data)
      })
    },

    onSuccess: (serverTask) => {
      // 5. 将权威的服务器响应写入详情缓存
      queryClient.setQueryData(taskKeys.detail(serverTask.id), serverTask)
    },

    onSettled: async () => {
      // 6. 最终用服务器数据重新校准列表
      await queryClient.invalidateQueries({ queryKey: taskKeys.lists() })
    },
  })
}
```

你会看到一个很有规律的事务式流程：

取消请求 → 保存快照 → 更新缓存 → 失败回滚 → 最终失效并重新验证。

这里的状态写入都发生在 Query 缓存中。没有必要把任务列表同步复制到 Zustand。

7.2 这份代码还没解决所有并发问题

以上版本适合学习“单次操作如何回滚”，但不是对任意并发场景都安全。考虑同一条任务连续发生两次修改：A 把标题改成“草稿”，B 随后改成“已发布”；如果 A 失败后用较早的快照回滚，就可能把 B 的新状态也覆盖掉。类似地，多个并发 mutation 的服务器响应可能乱序到达，最终状态未必等于用户最后一次操作。

这是最终一致性风险：客户端的临时预测、请求实际到达服务器的顺序、服务器接受写入的顺序与查询返回顺序，不一定完全一致。

应当根据业务复杂度选择策略：

1. **最简单：不用乐观改缓存。** 通过 `mutation.isPending` 显示“保存中”，成功后失效并刷新。对于低频表单，这是很好的默认方案。
2. **把临时结果放在 UI 层。** 例如根据 mutation 的变量临时显示一个正在更新的标签，不改共享缓存。这通常更容易回滚。
3. **控制同一资源的并发。** 对同一条记录的写入排队或禁用重复提交；服务端使用版本号、`If-Match` / ETag 或冲突检测。
4. **减少多次重复失效。** 当多个同类 mutation 同时执行时，可以只在最后一个仍未完成的 mutation 收尾时刷新列表：

```
onSettled: async () => {
  // 当前 mutation 仍在队列中，因此数量为 1 表示它可能是最后一个。
  if (queryClient.isMutating({ mutationKey: ['tasks', 'update'] }) === 1) {
    await queryClient.invalidateQueries({ queryKey: taskKeys.lists() })
  }
}
```

这只能减少不必要的并行刷新，**不能**自动解决服务器写入顺序、快照回滚互相覆盖或业务冲突。对于高并发协作编辑，必须把服务端一致性设计纳入考虑。

还有个常见陷阱：当任务从“未完成”改成“已完成”时，它应该从未完成列表消失、出现在已完成列表。最基本的乐观示例只是更新现有列表中的对象，并没有实现所有筛选集合的正确插入 / 移除。此时最安全的方式通常是成功或 settled 后失效所有相关列表，让服务器重新决定各个列表的成员关系；如果要手工维护列表，就必须正确处理筛选、排序、分页边界和总数。

第 8 章：分页、无限滚动、依赖查询和预取

8.1 分页：页码必须进入 query key

分页查询中，页码决定返回结果，所以它必须在 key 中。我们已经在 `taskKeys.list(params)` 中放入 `page`。组件改变页码后，Query 自然会切换到对应缓存条目。

```
import { keepPreviousData, useQuery } from '@tanstack/react-query'
import { getTasks } from './taskApi'
import { taskKeys } from './taskKeys'
import type { TaskListParams } from './taskTypes'

import * as React from 'react'

function PaginatedTasks() {
  const [page, setPage] = React.useState(1)
  const params: TaskListParams = { status: 'all', page, q: '' }

  const query = useQuery({
    queryKey: taskKeys.list(params),
    queryFn: ({ signal }) => getTasks(params, signal),
    placeholderData: keepPreviousData,
  })

  return (
    <section>
      {query.isPending ? <p>首次加载……</p> : null}
      {query.data && (
        <>
          {query.isPlaceholderData && <small>正在切换到新页，暂时保留上一页内容。</small>}
          <ul>{query.data.items.map((task) => <li key={task.id}>{task.title}</li>)}</ul>
          <button disabled={page <= 1} onClick={() => setPage((p) => p - 1)}>
            上一页
          </button>
          <button
            disabled={query.isPlaceholderData || !query.data.hasMore}
            onClick={() => setPage((p) => p + 1)}
          >
            下一页
          </button>
        </>
      )}
      {query.isFetching && <small>同步中……</small>}
      {query.isError && !query.data && <p role="alert">加载失败。</p>}
```

```
      </section>
    )
  }
```

`placeholderData: keepPreviousData` 可以在新页请求期间继续显示上一页数据，减少页面忽然跳回 loading 的闪烁。此时要关注 `isPlaceholderData`，例如在旧数据还在展示时，不要依据旧页的 `hasMore` 允许连续跳页。

8.2 无限滚动：页游标是 pageParam，不是新的 query key

无限滚动的目标是把下一页叠加到当前列表。与普通分页不同，所有页通常属于同一个 infinite query 缓存条目，下一页的游标由 `pageParam` 管理。

```
import { useInfiniteQuery } from '@tanstack/react-query'
import type { Task } from './taskTypes'
import { fetchJson } from '../../lib/http'

interface CursorPage {
  items: Task[]
  nextCursor: string | null
}

async function getTaskPage(cursor: string | null, signal?: AbortSignal) {
  const params = new URLSearchParams()
  if (cursor) params.set('cursor', cursor)
  return fetchJson<CursorPage>(`/api/tasks?${params}`, { signal })
}

function InfiniteTasks() {
  const query = useInfiniteQuery({
    queryKey: ['tasks', 'infinite', { status: 'all' }],
    initialPageParam: null as string | null,
    queryFn: ({ pageParam, signal }) => getTaskPage(pageParam, signal),
    getNextPageParam: (lastPage) => lastPage.nextCursor ?? undefined,
  })

  if (query.isPending) return <p>正在加载……</p>
  if (query.isError) return <p role="alert">加载失败: {query.error.message}</p>

  return (
    <section>
      {query.data.pages.flatMap((page) => page.items).map((task) => (
        <div key={task.id}>{task.title}</div>
      ))}
      <button
        disabled={!query.hasNextPage || query.isFetchingNextPage}
        onClick={() => query.fetchNextPage()}
      >
        {query.isFetchingNextPage ? '加载更多……' : query.hasNextPage ? '加载更多' : '没有更多了'}
      </button>
    </section>
  )
}
```

在 v5 中，`initialPageParam` 是必填项。`data.pages` 储存页面数据，`data.pageParams` 储存各页对应参数；`getNextPageParam` 返回 `null` 或 `undefined` 时表示没有下一页。注意 key 仍需包含会改变整组列表的筛选或搜索条件，但通常不把每个游标都加入 query key；游标由 infinite query 的 page 参数管理。

8.3 依赖查询：用 enabled 表达前置条件

有时候必须先查询用户，再用用户 ID 取得项目。不要在渲染函数里手动套一层 `await`；把依赖关系声明出来：

```
const userQuery = useQuery({
  queryKey: ['user', email],
  queryFn: ({ signal }) => getUserByEmail(email, signal),
})

const userId = userQuery.data?.id

const projectsQuery = useQuery({
  queryKey: ['projects', userId],
  queryFn: ({ signal }) => getProjectsByUserId(userId!, signal),
  enabled: Boolean(userId),
})
```

`enabled` 表示依赖条件还没满足时暂不自动执行该 query。第二个 query 在未启用时可能 `isPending: true`，但 `fetchStatus: 'idle'`，所以应理解为“等待依赖”，而不是网络正在加载。

依赖查询会形成请求瀑布：第二个请求必须等第一个结果回来才开始。如果 API 可以改造成一次返回用户及项目，或客户端已知用户 ID，就可以避免不必要的串行等待。

8.4 预取：在用户需要之前填充缓存

如果我们知道用户很可能会打开某个任务详情，可以在鼠标移入列表项时预取：

```
function TaskRow({ task }: { task: Task }) {
  const queryClient = useQueryClient()

  function prefetchDetail() {
    void queryClient.prefetchQuery({
      queryKey: taskKeys.detail(task.id),
      queryFn: ({ signal }) => getTask(task.id, signal),
      staleTime: 30_000,
    })
  }

  return (
    <button onMouseEnter={prefetchDetail} onFocus={prefetchDetail}>
      {task.title}
    </button>
  )
}
```

预取的关键不是“永远不加载”，而是让数据在真正打开详情页前已经进入对应 Query 缓存。详情页仍必须使用完全相同的 key，才能命中这一份数据。预取请求失败通常不会阻塞当前页面；详情页加载时仍应做好 loading / error 处理。

8.5 后台刷新与轮询

如果任务状态需要周期性刷新，可以设置轮询间隔：

```
useQuery({
  queryKey: taskKeys.list(params),
  queryFn: ({ signal }) => getTasks(params, signal),
  refetchInterval: 15_000,
  refetchIntervalInBackground: false,
})
```

轮询属于后台刷新策略，和 `staleTime` 不是同一个概念。不要对大量页面无差别地开高频轮询；应根据实时性需求、请求成本、页面可见性和服务端负载选择策略。

本章动手练习

给任务页面补齐分页、搜索和状态筛选，并验证以下原则：

- 改变搜索词时，query key 变化。
- 改变页码时，普通分页 query key 变化。
- 无限滚动的筛选条件进入 key，而下一页游标进入 pageParam。
- 预取和真正使用详情时采用完全相同的 key。

第 9 章：结构共享、引用稳定性与 Devtools

这些内容往往看起来像性能细节，但理解后可以避免很多莫名其妙的重复渲染。

9.1 结构共享是什么

TanStack Query 默认对 JSON 兼容的数据启用结构共享（structural sharing）。假设后台刷新返回了一个新任务数组，但 100 条任务里只有 1 条的标题变化，Query 会尽量保留未变化部分的对象引用。这样可以让依赖未变化数据的组件、`useMemo` 或 `useCallback` 更容易保持稳定。

一般不用关闭 `structuralSharing`。它主要针对 JSON 兼容结构；如果返回的数据含有特殊类实例、不可 JSON 化值或遇到大型响应的性能问题，再考虑官方文档中的配置方式和自定义策略。

9.2 query 结果对象和 data 引用不是一回事

`useQuery` 返回的结果对象本身不应被当作稳定引用；但其中的 `data` 会在适用情况下通过结构共享尽可能保持稳定。不要因为 `query` 对象每次渲染都被使用，就直接将整个结果对象放进依赖数组并期望引用永远不变。

更清晰的写法是只取真正需要的值：

```
const { data, isFetching } = useTasks(params)

const taskTitles = React.useMemo(
  () => data?.items.map((task) => task.title) ?? [],
```

```
    [data],
  )
```

但请记住，不要为了避免一次轻微重渲染而把代码弄得难以理解。先通过性能分析确认确实存在问题，再优化。

`select` 可用于在组件侧选择查询数据的一部分，减少组件订阅范围：

```
  const selectTitles = (page: TaskPage) => page.items.map((task) => task.title)

  const titlesQuery = useQuery({
    queryKey: taskKeys.list(params),
    queryFn: ({ signal }) => getTasks(params, signal),
    select: selectTitles,
  })
```

如果 selector 本身会创建新对象或数组，注意它的引用行为；可把稳定的 selector 放在组件外，必要时对计算结果做 memo 化。

9.3 使用 Query Devtools 学会看缓存

开发环境加入 `@tanstack/react-query-devtools`，可以查看 query key、数据、`status`、`fetchStatus`、是否 stale、最后更新时间和观察者数量。排查问题时，先回答：

1. 组件实际使用的 query key 是什么？
2. 同一 key 是否已经有数据？数据是 fresh 还是 stale？
3. 有没有另一个组件在订阅这个 key？
4. 是哪个事件触发了获取：挂载、聚焦、重新连接、失效、轮询，还是手动 refetch？
5. 请求函数是否抛出错误，是否正在进行重试？

这比在代码里到处打印 `console.log` 更容易理解缓存的整体生命周期。

第 10 章：Zustand 入门——store、action 与 selector

到这里，任务列表已经由 Query 管理。接下来我们真正需要 Zustand：保存当前筛选条件、搜索输入、选中的任务和侧栏开关。

10.1 创建第一个 store

```ts
// src/stores/taskUiStore.ts
import { create } from 'zustand'
import type { TaskStatus } from '../features/tasks/taskTypes'

interface TaskUiState {
  status: TaskStatus
  search: string
  selectedTaskId: string | null
  sidebarOpen: boolean
  setStatus: (status: TaskStatus) => void
  setSearch: (search: string) => void
```

```
  selectTask: (id: string | null) => void
  toggleSidebar: () => void
}

export const useTaskUiStore = create<TaskUiState>()((set) => ({
  status: 'all',
  search: '',
  selectedTaskId: null,
  sidebarOpen: true,
  setStatus: (status) => set({ status, selectedTaskId: null }),
  setSearch: (search) => set({ search }),
  selectTask: (selectedTaskId) => set({ selectedTaskId }),
  toggleSidebar: () => set((state) => ({ sidebarOpen: !state.sidebarOpen })),
}))
```

Zustand 的 `create` 返回一个 React hook。store 对象同时包含状态和 action（更新状态的方法）。这里的 action 只是普通函数，但把它们放在 store 中，可以让多个组件以相同方式触发业务状态变化。

`set` 默认会对顶层状态进行浅合并，因此 `set({ search })` 就足够更新一个顶层字段。但嵌套对象只浅合并一层，更新嵌套属性时必须显式复制嵌套对象，不能直接修改原有对象：

```
set((state) => ({
  preferences: {
    ...state.preferences,
    density: 'compact',
  },
}))
```

10.2 Selector：只订阅你需要的字段

```
function SidebarButton() {
  const sidebarOpen = useTaskUiStore((state) => state.sidebarOpen)
  const toggleSidebar = useTaskUiStore((state) => state.toggleSidebar)

  return (
    <button onClick={toggleSidebar}>
      {sidebarOpen ? '收起侧栏' : '展开侧栏'}
    </button>
  )
}
```

selector 的作用是让组件订阅 store 的某个结果，而不是整个 store。`search` 改变时，只选择 `sidebarOpen` 的组件通常不会因为这个变化而重渲染。

初学者应该避免如下写法：

```
// 不推荐：组件订阅整个 store 的所有字段
const store = useTaskUiStore()
```

优先只取当前组件需要的字段和 action。如果需要一次选择多个值，直接返回一个新对象可能因为引用变化导致不必要的渲染；可以用官方提供的 `useShallow`：

```
import { useShallow } from 'zustand/react/shallow'

const { status, search, setStatus } = useTaskUiStore(
```

```
    useShallow((state) => ({
      status: state.status,
      search: state.search,
      setStatus: state.setStatus,
    })),
  )
```

10.3 Derived state：能算出来就不要重复保存

假设页面有 `items`，同时又维护 `visibleItems`，每次 items、search、status 变化时都必须记得更新它。这会增加同步遗漏的可能。更好的办法是从已有状态推导结果。

如果筛选发生在客户端：

```
const items = query.data?.items ?? []
const visibleItems = React.useMemo(
  () => items.filter((task) => task.title.toLowerCase().includes(search.toLowerCase())),
  [items, search],
)
```

但如果服务器负责搜索或状态筛选，筛选条件就应该作为 query 参数进入 key：

```
const status = useTaskUiStore((state) => state.status)
const search = useTaskUiStore((state) => state.search)
const page = 1

const query = useTasks({ status, q: search, page })
```

这段代码没有把任务数据复制到 Zustand，只是让 UI 状态决定当前订阅哪一个 Query 缓存。用户输入发生变化，key 相应变化，Query 自动切换到正确的查询结果。

10.4 多个 store 应该如何拆分？

不要把“每个字段一个 store”当作性能秘诀，也不要无条件做一个巨大 store。一个实用的拆分原则是按职责和业务域拆：

- `useTaskUiStore`：任务页面筛选、选中项和面板状态。
- `useAuthUiStore`：登录流程中的本地交互状态（注意真正认证凭据仍按安全架构处理）。
- `useEditorStore`：复杂编辑器里的临时工作区状态。

如果某块状态只被单个组件使用，用 React `useState` 往往最简单；跨多个不相邻组件共享且生命周期合适时，再考虑 Zustand。拆分后可以减少无关订阅，也更容易测试和理解。

不要创建 `useTasksStore` 专门存一份从服务器下载的任务列表。 列表、详情等仍然由 Query 维护，Zustand 只负责与任务页面有关的本地选择与交互。

第 11 章：把 Zustand 状态和 Query 连接起来，但不互相复制

下面把前面的设计整合成一个页面。Zustand 拥有筛选器和搜索词；TanStack Query 拥有对应筛选结果的缓存。

```tsx
function TasksPage() {
  const status = useTaskUiStore((state) => state.status)
  const search = useTaskUiStore((state) => state.search)
  const setStatus = useTaskUiStore((state) => state.setStatus)
  const setSearch = useTaskUiStore((state) => state.setSearch)
  const selectedTaskId = useTaskUiStore((state) => state.selectedTaskId)
  const selectTask = useTaskUiStore((state) => state.selectTask)

  const query = useTasks({ status, q: search, page: 1 })

  return (
    <main>
      <label>
        搜索任务
        <input value={search} onChange={(event) => setSearch(event.target.value)} />
      </label>

      <label>
        任务状态
        <select value={status} onChange={(event) => setStatus(event.target.value as TaskStatus)}>
          <option value="all">全部</option>
          <option value="open">未完成</option>
          <option value="done">已完成</option>
        </select>
      </label>

      {query.isPending && <p>加载中……</p>}
      {query.isError && !query.data && <p role="alert">{query.error.message}</p>}
      {query.isFetching && <small>同步中……</small>}

      <ul>
        {query.data?.items.map((task) => (
          <li key={task.id}>
            <button onClick={() => selectTask(task.id)}>
              {task.id === selectedTaskId ? '▶ ' : ''}{task.title}
            </button>
          </li>
        ))}
      </ul>

      {selectedTaskId && <TaskDetails taskId={selectedTaskId} />}
    </main>
  )
}

function TaskDetails({ taskId }: { taskId: string }) {
  const query = useQuery({
    queryKey: taskKeys.detail(taskId),
    queryFn: ({ signal }) => getTask(taskId, signal),
  })

  if (query.isPending) return <p>正在读取详情……</p>
  if (query.isError && !query.data) return <p role="alert">{query.error.message}</p>
  if (!query.data) return null
```

```
  return <article><h2>{query.data.title}</h2></article>
}
```

这个页面最值得观察的是：`selectedTaskId` 来自 Zustand，但任务详情对象没有存入 Zustand。组件根据 ID 使用 Query 读取详情。这样，选择状态和服务器对象各自只有一个负责人。

实际项目还需做输入防抖、空搜索处理、URL 同步等体验优化。但架构原则不变：**用 UI 状态决定查询参数，用 Query 管理查询结果。**

第 12 章：Zustand persist——些得留下？

刷新页面后侧栏状态或用户偏好是否要保留？这时可以使用 Zustand 的 `persist` middleware。它用于将 store 的部分或全部状态持久化到存储中，默认常见用法是 `localStorage`。

12.1 部分持久化：partialize

下面的代码是对第 10 章 `taskUiStore.ts` 的升级版，请用它替换原文件中的 store 定义，不要同时创建两个同名 store。并不是 store 中所有字段都应该存下来。例如“当前选中任务 ID”可能只属于当前页面会话，“侧栏是否展开”则可能是可保留的偏好。

```
import { create } from 'zustand'
import { createJSONStorage, persist } from 'zustand/middleware'
import type { TaskStatus } from '../features/tasks/taskTypes'

interface TaskUiState {
  status: TaskStatus
  search: string
  selectedTaskId: string | null
  sidebarOpen: boolean
  setStatus: (status: TaskStatus) => void
  setSearch: (search: string) => void
  selectTask: (id: string | null) => void
  toggleSidebar: () => void
}

type PersistedTaskUi = Pick<TaskUiState, 'status' | 'sidebarOpen'>

export const useTaskUiStore = create<TaskUiState>()(
  persist(
    (set) => ({
      status: 'all',
      search: '',
      selectedTaskId: null,
      sidebarOpen: true,
      setStatus: (status) => set({ status, selectedTaskId: null }),
      setSearch: (search) => set({ search }),
      selectTask: (selectedTaskId) => set({ selectedTaskId }),
      toggleSidebar: () => set((state) => ({ sidebarOpen: !state.sidebarOpen })),
    }),
```

```
    {
      name: 'task-ui-v2',
      storage: createJSONStorage(() => localStorage),
      partialize: (state): PersistedTaskUi => ({
        status: state.status,
        sidebarOpen: state.sidebarOpen,
      }),
      version: 2,
    },
  ),
)
```

`partialize` 决定哪些字段写入持久化存储，但运行时 store 仍然可以拥有其他字段。这里不会持久化搜索词和选中 ID。

12.2 版本升级与 migrate

当存储结构变更时，例如旧版本字段名是 `isSidebarOpen`，新版本改成 `sidebarOpen`，就需要版本迁移。`version` 用于表示持久化格式版本；版本不匹配时可用 `migrate` 把旧对象转为新结构。

```
{
  name: 'task-ui-v2',
  version: 2,
  migrate: (persistedState, version) => {
    if (version === 1) {
      const old = persistedState as {
        isSidebarOpen?: boolean
        status?: TaskStatus
      }
      return {
        sidebarOpen: old.isSidebarOpen ?? true,
        status: old.status ?? 'all',
      }
    }
    return persistedState as PersistedTaskUi
  },
  partialize: (state): PersistedTaskUi => ({
    status: state.status,
    sidebarOpen: state.sidebarOpen,
  }),
}
```

迁移逻辑应验证字段、提供合理默认值，不要无条件相信浏览器存储里的内容。官方文档特别提醒：JSON 存储解析本身不进行运行时 schema 验证，损坏或被篡改的数据可能在运行时被直接当作目标类型使用。对重要持久化结构，可以增加验证器（例如 Zod），验证失败则回退默认状态。

12.3 不要把秘密放进持久化 store

不要把密码、访问令牌、长期凭据、私钥等秘密放入 `localStorage`。浏览器中的客户端 JavaScript 可以读取它们，发生 XSS 时尤其危险。`persist` 不是加密保险箱，也不等于安全存储。

此外，持久化只应针对确实需要跨刷新保留的客户端偏好。服务端数据不能因为“刷新后还要看到”就复制进 Zustand 并持久化；如果确实需要持久化 Query 缓存，应使用 TanStack Query 相应的持久化方案并理解过期、版本、用户隔离和敏感数据的安全含义，而不是另建一份业务数据仓库。

对于 SSR，持久化 rehydrate 可能在客户端晚于首次渲染执行，导致服务端与客户端初始 UI 不一致。需要时参考 Zustand 的 `skipHydration` 与手动 rehydrate 方案，不要简单假设 `localStorage` 在服务端渲染时可用。

第 13 章：测试与调试——让每个用例都独立

13.1 为什么每个测试要隔离 QueryClient

QueryClient 有缓存和默认行为。如果多个测试共享同一个实例，前一个测试留下的数据可能让后一个测试不再发请求；缓存状态、重试和后台刷新也可能让测试结果相互影响。因此，最稳妥的策略是**每个测试创建新的 QueryClient**，并在测试里关闭重试以避免失败场景等待多次重试。

以下用 Vitest + React Testing Library 风格示例：

```
import type { PropsWithChildren } from 'react'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { render, screen, waitFor } from '@testing-library/react'
import { describe, expect, it, vi } from 'vitest'

function createTestQueryClient() {
  return new QueryClient({
    defaultOptions: {
      queries: {
        retry: false,
        gcTime: Infinity,
      },
      mutations: { retry: false },
    },
  })
}

function createWrapper(client: QueryClient) {
  return function Wrapper({ children }: PropsWithChildren) {
    return (
      <QueryClientProvider client={client}>
        {children}
      </QueryClientProvider>
    )
  }
}

describe('任务列表', () => {
  it('成功后展示服务端任务', async () => {
    const client = createTestQueryClient()
    // 在实际项目中推荐使用 MSW 拦截网络请求，避免真的访问后端。
    // 这里假设测试组件依赖的 API 已经被 mock 成返回任务列表。
```

```
    render(<TaskList params={{ status: 'all', page: 1, q: '' }} />, {
      wrapper: createWrapper(client),
    })

    expect(await screen.findByText('写教程')).toBeInTheDocument()
  })
})
```

上面是测试架构示例，`TaskList` 与 API mock 需要按项目实际代码准备。生产级组件测试推荐用 MSW 统一模拟 HTTP 层，测试成功、失败、空数据和慢请求；不建议把测试建立在“某个共享缓存刚好是空的”这一假设上。

13.2 测试 Zustand 时也要重置状态

如果被测 store 是模块级单例，测试间的状态会保留。可以在每个测试前重置到初始状态：

```
beforeEach(() => {
  useTaskUiStore.setState(useTaskUiStore.getInitialState(), true)
})
```

第二个参数 `true` 表示替换状态，而不仅是浅合并。请在真实测试代码中确保导入了正确的 store，并处理 `persist` 对 localStorage 的影响，例如每个测试前清空对应存储键或使用隔离的 storage mock。另一种方式是在需要强隔离的场景创建 store factory，而不是共享模块级实例。

13.3 常见问题的排查顺序

问题 A：切换筛选后却显示上一种筛选的数据。

先看 query key 是否包含筛选条件；再看 `queryFn` 取的是不是同一个变量。不要先通过手动清空 Zustand 数据来“修复”。

问题 B：页面回到前台就发送请求。

看 query 是否 stale，以及 `refetchOnWindowFocus` 是否启用。默认行为如此并不一定是 bug。根据新鲜度需求调整 `staleTime` 或特定 refetch 选项。

问题 C：Mutation 成功了，其他列表没变化。

检查 `onSuccess` / `onSettled` 是否对相关 query 调用了 `invalidateQueries`；或者服务器响应是否适合直接 `setQueryData` 写入精确缓存。

问题 D：失败回滚覆盖了另一条新操作。

这是乐观更新的并发问题，不是简单的 React 渲染问题。检查快照范围、同一实体的并发提交、回滚是否覆盖后续写入、是否应该序列化请求，以及服务端是否需要版本冲突检查。

问题 E：某个组件无关状态变化也频繁重渲染。

Zustand 检查 selector 是否选择整个 store 或每次创建新对象；多值选择时尝试 `useShallow`。Query 检查组件依赖的是整个 query 返回对象还是稳定的 `data` 部分，再用 React Profiler 验证。

第 14 章：用一张规则表收束整套心智模型

你遇到的问题	首先考虑的工具 / API	不要默认采用的做法
GET 数据、缓存、加载错误和后台刷新	useQuery、query key、queryFn	手写一套 useEffect + useState 缓存
写服务器、提交表单、创建和删除	useMutation	在多个组件里各自 fetch 并同步数据
写入后让读取结果同步	invalidateQueries、必要时 setQueryData	清空整个缓存或把数据复制到 Zustand
定义数据新鲜度	staleTime	误以为 stale 就等于立即请求
决定闲置数据保留时间	gcTime	把 gcTime 当成刷新间隔
分页和筛选变化	把所有影响响应的参数放进 query key	多个不同请求共用同一个 key
高即时反馈的修改	乐观缓存更新或 UI 层临时显示	忽略失败回滚和并发覆盖
临时选择、展开状态、工作区状态	React state 或 Zustand	把它们伪装成服务器查询
多组件共享客户端状态	Zustand selector 与 action	每个组件都订阅整个 store
需要记住偏好	persist + partialize + 版本迁移	把所有状态甚至秘密全部写入 localStorage
测试 query hook / 组件	每个测试建立独立 QueryClient	测试间共享缓存导致偶发失败

自测：不看上文也能解释吗？

请用自己的话回答以下问题。只要有一个问题说不清，就回到对应章节实际敲代码。

1. Query key 为什么不仅是一个名字？若搜索词不在 key 中，会发生什么？
2. staleTime: 60_000 和 gcTime: 60_000 各自表达什么？一个 query 变 stale 后是否一定立刻发请求？
3. isPending、isFetching 和 isRefetchError 分别应该怎样影响 UI？
4. invalidateQueries、refetch、setQueryData 的语义有什么不同？
5. Mutation 的 onMutate、onError、onSuccess、onSettled 各适合干什么？
6. 乐观更新为什么要先取消相关 query 并保存快照？为什么普通快照回滚仍可能在并发时出错？
7. 普通分页与无限滚动，页码 / 游标分别应该放在什么位置？
8. Zustand selector 如何帮助细粒度订阅？为什么派生数据通常不需要另存一份？
9. partialize、version 与 migrate 分别解决什么问题？为什么不能持久化访问令牌？
10. 为什么每个测试创建独立 QueryClient？为什么不应把服务器任务列表同时保存在 Query 和 Zustand 中？

最后的项目实践任务

从空白页面做出一个小型任务管理页面，并依次验收：

1. 查询全部任务，显示首次 loading、错误和已有数据的后台刷新状态。
2. 实现带筛选、搜索和页码的列表，保证每个响应变量都进入 query key。
3. 添加任务和修改任务，成功后刷新相关列表；详情缓存尽量使用服务器返回对象更新。
4. 对“标记完成”做一次乐观更新，添加失败回滚；再模拟快速连续点击，观察并发风险。
5. 添加详情预取，并用 Devtools 检查 query key、fresh / stale 状态和请求触发来源。
6. 用 Zustand 保存筛选器、选中项和侧栏状态；只持久化你明确希望在刷新后保留的偏好。
7. 为成功、失败与状态隔离写测试。

做到这里，你才算真正从零到一掌握了基础。接下来可以深入官方文档中的 SSR / Hydration、Query 持久化、复杂并发更新、错误边界和 Zustand slices，但应该在基本缓存模型和状态边界清晰之后再学习。

官方文档与延伸阅读

以下链接均指向官方文档。文档 API 可能随版本变化；遇到选项、回调参数或 SSR 行为不确定时，应以你项目锁定版本对应的文档为准。

TanStack Query（React v5）

1. Queries：`useQuery` 与查询状态
2. Query Keys：查询键设计
3. Query Functions：请求函数
4. Important Defaults：默认行为、新鲜度与结构共享
5. Query Cancellation：取消查询
6. Mutations：写操作生命周期
7. Invalidation from Mutations：写后失效
8. Optimistic Updates：乐观更新
9. Paginated / Lagged Queries：分页与保留旧数据
10. Infinite Queries：无限滚动
11. Dependent Queries：依赖查询
12. Prefetching & Router Integration：预取
13. Render Optimizations：渲染优化
14. Testing：测试

Zustand

1. Introduction：安装与入门
2. Updating State：更新状态
3. Immutable State and Merging：不可变更新与浅合并
4. Prevent Rerenders with useShallow：细粒度订阅与减少重渲染
5. Slices Pattern：拆分大型 store
6. persist middleware：持久化、partialize、version 与 migrate
7. Testing：测试 store 与组件